



中山大學
SUN YAT-SEN UNIVERSITY

结构体进阶与练习

學大山中立國
中山大学计算机学院



讲课人: 万海

目录

CONTENTS

01

计算距离和

02

计算时间差

03

停车系统

04

银行系统

05



中山大學

SUN YAT-SEN UNIVERSITY

例题一-计算距离和

计算两个距离（英尺，英寸）的和。

示例1：输入第一个距离：23英尺8.6英寸；

输入第二个距离：33英尺2.4英寸。

输出：57英尺11英寸

结构体要一个个输入输出

```
1.c
1 #include <stdio.h>
2
3 struct Distance {
4     int feet;
5     float inch;
6 } d1, d2, result;
7
8 int main() {
9     // take first distance input
10    printf("Enter 1st distance\n");
11    printf("Enter feet: ");
12    scanf("%d", &d1.feet);
13    printf("Enter inch: ");
14    scanf("%f", &d1.inch);
15
16    // take second distance input
17    printf("\nEnter 2nd distance\n");
18    printf("Enter feet: ");
19    scanf("%d", &d2.feet);
20    printf("Enter inch: ");
21    scanf("%f", &d2.inch);
22
23    // adding distances
24    result.feet = d1.feet + d2.feet;
25    result.inch = d1.inch + d2.inch;
26
27    // convert inches to feet if greater than 12
28    while (result.inch >= 12.0) {
29        result.inch = result.inch - 12.0;
30        ++result.feet;
31    }
32    printf("\nSum of distances = %d\'-%.1f\"", result.feet, result.inch);
33    return 0;
34 }
```



例题一-计算距离和

```
struct Distance {  
    int feet;  
    float inch;  
} d1, d2, result;
```

定义距离

```
// adding distances  
result.feet = d1.feet + d2.feet;  
result.inch = d1.inch + d2.inch;
```

分别计算英尺和英寸部分

```
// convert inches to feet if greater than 12
```

```
while (result.inch >= 12.0) {  
    result.inch = result.inch - 12.0;  
    ++result.feet;  
}
```

检查是否进位

```
printf("\nSum of distances = %d\'-%.1f\'", result.feet, result.inch);  
return 0;
```



中山大學

SUN YAT-SEN UNIVERSITY

例题-计算时间差

计算两个时间（时分秒格式）的差（开始时间 - 结束时间）。

示例1：输入开始的时分秒：13时34分55秒；

输入结束的时分秒：8时12分15秒。

输出：（13:34:55 - 8:12:15 = ） 5:22:40

```
2.c
1  #include <stdio.h>
2
3  struct TIME {
4      int seconds;
5      int minutes;
6      int hours;
7  };
8
9  void differenceBetweenTimePeriod(struct TIME t1,
10                                  struct TIME t2,
11                                  struct TIME *diff);
12
13 int main() {
14     struct TIME startTime, stopTime, diff;
15
16     printf("Enter the start time. \n");
17     printf("Enter hours, minutes and seconds: ");
18     scanf("%d %d %d", &startTime.hours,
19             &startTime.minutes,
20             &startTime.seconds);
21
22     printf("Enter the stop time. \n");
23     printf("Enter hours, minutes and seconds: ");
24     scanf("%d %d %d", &stopTime.hours,
25             &stopTime.minutes,
26             &stopTime.seconds);
27
28     // Difference between start and stop time
29     differenceBetweenTimePeriod(startTime, stopTime, &diff);
30     printf("\nTime Difference: %d:%d:%d - ", startTime.hours,
31           startTime.minutes,
32           startTime.seconds);
33     printf("%d:%d:%d ", stopTime.hours,
34           stopTime.minutes,
35           stopTime.seconds);
36     printf("\n= %d:%d:%d\n", diff.hours,
37           diff.minutes,
38           diff.seconds);
39     return 0;
40 }
41
42 // Computes difference between time periods
43 void differenceBetweenTimePeriod(struct TIME start,
44                                  struct TIME stop,
45                                  struct TIME *diff) {
46     while (stop.seconds > start.seconds) {
47         --start.minutes;
48         start.seconds += 60;
49     }
50     diff->seconds = start.seconds - stop.seconds;
51     while (stop.minutes > start.minutes) {
52         --start.hours;
53         start.minutes += 60;
54     }
55     diff->minutes = start.minutes - stop.minutes;
56     diff->hours = start.hours - stop.hours;
57 }
```

还没有
定义
指针



中山大學

SUN YAT-SEN UNIVERSITY

例题2-计算时间差

```
#include <stdio.h>
```

```
struct TIME {  
    int seconds;  
    int minutes;  
    int hours;  
};
```

定义时间结构体

```
void differenceBetweenTimePeriod(struct TIME t1,  
                                 struct TIME t2,  
                                 struct TIME *diff);
```

差要用
传引用

信息不会
保留给主函数

结构体名字本身是变量, 不是地址



例题2-计算时间差

```
// Computes difference between time periods
void differenceBetweenTimePeriod(struct TIME start,
                                struct TIME stop,
                                struct TIME *diff) {
    while (stop.seconds > start.seconds) {
        --start.minutes;
        start.seconds += 60;
    }
    diff->seconds = start.seconds - stop.seconds;
    while (stop.minutes > start.minutes) {
        --start.hours;
        start.minutes += 60;
    }
    diff->minutes = start.minutes - stop.minutes;
    diff->hours = start.hours - stop.hours;
}
```

如果秒部分不够减，进行借位

如果分钟部分不够减，进行借位



中山大学

SUN YAT-SEN UNIVERSITY

例题B-停车系统

请你给一个停车场设计一个停车系统。停车场总共有三种不同大小的车位：大，中和小，每种尺寸分别有固定数目的车位。

请你实现 `ParkingSystem` 类：

- `ParkingSystem(int big, int medium, int small)` 初始化 `ParkingSystem` 类，三个参数分别对应每种停车位的数目。
- `bool addCar(int carType)` 检查是否有 `carType` 对应的停车位。 `carType` 有三种类型：大，中，小，分别用数字 1， 2 和 3 表示。一辆车只能停在 `carType` 对应尺寸的停车位中。如果没有空车位，请返回 `false`，否则将该车停入车位并返回 `true`。

示例 1：

输入：

["ParkingSystem", "addCar", "addCar", "addCar", "addCar"]

[[1, 1, 0], [1], [2], [3], [1]]

输出：

[null, true, true, false, false]

```
3c
3 #include <stdlib.h>
4
5 typedef struct {
6     int b, m, s;
7 } ParkingSystem;
8
9 ParkingSystem* parkingSystemCreate(int big, int medium, int small) {
10     ParkingSystem* ret = malloc(sizeof(ParkingSystem));
11     ret->b = big;
12     ret->m = medium;
13     ret->s = small;
14     return ret;
15 }
16
17 bool parkingSystemAddCar(ParkingSystem* obj, int carType) {
18     if (carType == 1) {
19         if (obj->b > 0) {
20             obj->b--;
21             return true;
22         }
23     } else if (carType == 2) {
24         if (obj->m > 0) {
25             obj->m--;
26             return true;
27         }
28     } else if (carType == 3) {
29         if (obj->s > 0) {
30             obj->s--;
31             return true;
32         }
33     }
34     return false;
35 }
36
37 void parkingSystemFree(ParkingSystem* obj) {
38     free(obj);
39 }
40
41 int main() {
42     //char *action[50] = {"ParkingSystem", "addCar", "addCar", "addCar", "addCar"};
43     int N = 5;
44     long data[5][3] = {{1, 1, 0}, {1}, {2}, {3}, {1}};
45     ParkingSystem* obj = parkingSystemCreate(data[0][0], data[0][1], data[0][2]);
46     for(int i=1; i<N; i++){
47         bool param_1 = parkingSystemAddCar(obj, data[i][0]);
48         if(param_1) printf("true\n"); else printf("false\n");
49     }
50     parkingSystemFree(obj);
51     return 0;
52 }
53 }
```




例题B-停车系统

```
typedef struct {  
    int b, m, s;  
} ParkingSystem;
```

定义停车系统

```
ParkingSystem* parkingSystemCreate(int big, int medium, int small) {  
    ParkingSystem* ret = malloc(sizeof(ParkingSystem));  
    ret->b = big;  
    ret->m = medium;  
    ret->s = small;  
    return ret;  
}
```

构造一个停车系统



例题B-停车系统

```
bool parkingSystemAddCar(ParkingSystem* obj, int carType) {  
    if (carType == 1) {  
        if (obj->b > 0) {  
            obj->b--;  
            return true;  
        }  
    } else if (carType == 2) {  
        if (obj->m > 0) {  
            obj->m--;  
            return true;  
        }  
    } else if (carType == 3) {  
        if (obj->s > 0) {  
            obj->s--;  
            return true;  
        }  
    }  
    return false;  
}
```

减少对应停车位数量

```
void parkingSystemFree(ParkingSystem* obj) {  
    free(obj);  
}
```



中山大學

SUN YAT-SEN UNIVERSITY

例题一-银行系统

你的任务是为一个很受欢迎的银行设计一款程序，以自动化执行所有传入的交易（转账，存款和取款）。银行共有 n 个账户，编号从 1 到 n 。每个账号的初始余额存储在一个下标从 0 开始的整数数组 `balance` 中，其中第 $(i + 1)$ 个账户的初始余额是 `balance[i]`。

请你执行所有 **有效的** 交易。如果满足下面全部条件，则交易 **有效**：

- 指定的账户数量在 1 和 n 之间，且
- 取款或者转账需要的钱的总数 **小于或者等于** 账户余额。

实现 `Bank` 类：

- `Bank(long[] balance)` 使用下标从 0 开始的整数数组 `balance` 初始化该对象。
- `boolean transfer(int account1, int account2, long money)` 从编号为 `account1` 的账户向编号为 `account2` 的账户转账 `money` 美元。如果交易成功，返回 `true`，否则，返回 `false`。
- `boolean deposit(int account, long money)` 向编号为 `account` 的账户存款 `money` 美元。如果交易成功，返回 `true`；否则，返回 `false`。
- `boolean withdraw(int account, long money)` 从编号为 `account` 的账户取款 `money` 美元。如果交易成功，返回 `true`；否则，返回 `false`。

示例：

输入：

```
["Bank", "withdraw", "transfer", "deposit", "transfer", "withdraw"]  
[[[10, 100, 20, 50, 30]], [3, 10], [5, 1, 20], [5, 20], [3, 4, 15], [10, 50]]
```

输出：

```
[null, true, true, true, false, false]
```



例题-银行系统

```
typedef struct {
    long long * balance;
    int balanceSize;
} Bank;

Bank* bankCreate(long long* balance, int balanceSize) {
    Bank * obj = (Bank *)malloc(sizeof(Bank));
    obj->balance = (long long *)malloc(sizeof(long long) * balanceSize);
    obj->balanceSize = balanceSize;
    memcpy(obj->balance, balance, sizeof(long long) * balanceSize);
    return obj;
}
```

定义和构造银行系统

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <memory.h>
5
6 typedef struct
7 {
8     long long * balance;
9     int balanceSize;
10 } Bank;
11
12 // Bank bankCreate(long long* balance, int balanceSize) {
13 //     Bank * obj = (Bank *)malloc(sizeof(Bank));
14 //     obj->balance = (long long *)malloc(sizeof(long long) * balanceSize);
15 //     obj->balanceSize = balanceSize;
16 //     memcpy(obj->balance, balance, sizeof(long long) * balanceSize);
17 //     return obj;
18 // }
19
20 // Bank bankWithdraw(Bank* obj, int account, int amount, long long money) {
21 //     if (account < 0 || account > obj->balanceSize - 1 || obj->balance[account] - amount < 0) {
22 //         return false;
23 //     }
24 //     if (amount > money) {
25 //         return false;
26 //     }
27 //     obj->balance[account] -= amount;
28 //     return true;
29 // }
30
31 // Bank bankDeposit(Bank* obj, int account, long long money) {
32 //     if (account < 0 || account > obj->balanceSize - 1) {
33 //         return false;
34 //     }
35 //     obj->balance[account] += money;
36 //     return true;
37 // }
38
39 // Bank bankTransfer(Bank* obj, int account, long long money) {
40 //     if (account < 0 || account > obj->balanceSize - 1 || money < 0) {
41 //         return false;
42 //     }
43 //     obj->balance[account] -= money;
44 //     return true;
45 // }
46
47 // Bank bankFree(Bank* obj) {
48 //     free(obj->balance);
49 //     free(obj);
50 // }
```

```
51 int main() {
52     int N = 6;
53     const char action[10][10] = {"Bank", "withdraw", "transfer", "deposit", "transfer", "withdraw"};
54     long long data[10][10] = {{10, 100, 20, 50, 30}, {3, 10}, {5, 1, 20}, {5, 20}, {3, 4, 15}, {10, 50}};
55     Bank obj = bankCreate(data[0], 5);
56     for (int i = 1; i < N; i++) {
57         bool param_1 = 0;
58         // printf("Exit", action[i]);
59         if (strcmp(action[i], "withdraw") == 0) {
60             param_1 = bankWithdraw(obj, data[i][0], data[i][1]);
61             printf("withdraw ");
62         }
63         else if (strcmp(action[i], "transfer") == 0) {
64             param_1 = bankTransfer(obj, data[i][0], data[i][1], data[i][2]);
65             printf("transfer ");
66         }
67         else if (strcmp(action[i], "deposit") == 0) {
68             param_1 = bankDeposit(obj, data[i][0], data[i][1]);
69             printf("deposit ");
70         }
71         if (param_1) printf("true\n"); else printf("false\n");
72     }
73     bankFree(obj);
74     return 0;
75 }
```



例题-银行系统

```
bool bankTransfer(Bank* obj, int account1, int account2, long long money) {  
    if (account1 > obj->balanceSize || account2 > obj->balanceSize || obj->balance[account1 - 1] < money) {  
        return false;  
    }  
    obj->balance[account1 - 1] -= money;  
    obj->balance[account2 - 1] += money;  
    return true;  
}
```

涉及多种动作：两账户交易

```
bool bankDeposit(Bank* obj, int account, long long money) {  
    if (account > obj->balanceSize) {  
        return false;  
    }  
    obj->balance[account - 1] += money;  
    return true;  
}
```

存款

```
bool bankWithdraw(Bank* obj, int account, long long money) {  
    if (account > obj->balanceSize || obj->balance[account - 1] < money) {  
        return false;  
    }  
    obj->balance[account - 1] -= money;  
    return true;  
}
```

取款

```
void bankFree(Bank* obj) {  
    free(obj->balance);  
}
```



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

學大山中立國

中山大学计算机学院



编制人: 课题组